

The Vanka Smoother

Theory, and a high-performance implementation for the SFEM
semi-structured matrix-free CVFEM Navier–Stokes solver

September 5, 2026

Why this document exists

Measured on our solver, the block-Jacobi smoother has an asymptotic convergence factor of $\rho \approx 0.981$ at $\text{Re} = 1$ — 1.9% error removal per sweep, 120 sweeps per decade — and *diverges* for $\omega \geq 0.5$. The SIMPLE variant measures identically. Local Fourier analysis for Stokes puts a Vanka smoother at $\rho = 0.08$.

That difference, roughly a factor of 50 in iteration count, is the single largest lever available to this solver.

1 What Vanka is, and why block-Jacobi fails here

1.1 The saddle-point structure

Our linearised system has the block form

$$\mathcal{A} = \begin{pmatrix} F & B^\top \\ B & -C \end{pmatrix}, \quad \begin{pmatrix} \delta u \\ \delta p \end{pmatrix}, \quad (1)$$

where F is the momentum operator (viscous plus convective), B the discrete divergence, $B^\top$ the pressure gradient, and C the Rhie–Chow stabilisation. Without stabilisation $C = 0$ and the system is a genuine saddle point.

A *point-block* smoother inverts, at each node i , the 4×4 block

$$D_i = \begin{pmatrix} F_{ii} & B_{ii}^\top \\ B_{ii} & -C_{ii} \end{pmatrix}, \quad (2)$$

and this is where it fails. The pressure row of D_i contains only the *local* part of the divergence constraint. Continuity is a statement about fluxes across the faces *surrounding* a node, so B_{ii} carries almost none of it: the constraint that actually determines the pressure lives in the off-diagonal couplings B_{ij} , $j \neq i$, which the point-block smoother discards entirely. Damping the momentum equation while ignoring the constraint is why $\rho \rightarrow 1$: the velocity error is smoothed, and the pressure error is barely touched.

1.2 The Vanka idea

Vanka’s construction (S. P. Vanka, *JCP* **65** (1986) 138–158) replaces the per-node block by a small *coupled local system* over a patch Ω_k of degrees of freedom chosen so that at least one pressure unknown appears *together with all the velocity unknowns that its continuity equation references*. Writing R_k for the restriction onto patch k , one sweep of additive Vanka is

$$x \leftarrow x + \omega \sum_k R_k^\top (R_k \mathcal{A} R_k^\top)^{-1} R_k (b - \mathcal{A}x). \quad (3)$$

The inverted object $\mathcal{A}_k = R_k \mathcal{A} R_k^\top$ is the true operator restricted to the patch, *including* the velocity–pressure coupling. That is the whole difference from block-Jacobi, and it is why the local solve can enforce the constraint.

Published rates (Stokes)

Smoother	ρ (periodic)	ρ (Dirichlet)
Vanka	0.08	0.10
Triad Gauss–Seidel	0.26	0.36
Triad Jacobi	0.49	divergent
<i>our point-block Jacobi (measured, Re=1)</i>	—	<i>0.981</i>

Claus & Bolten, [arXiv:2008.08719](https://arxiv.org/abs/2008.08719). Our smoother is weaker than the entry that study calls divergent, under the boundary conditions we run.

2 Choosing the patch for a colocated CVFEM discretisation

Vanka was devised for staggered (MAC) grids, where the patch is obvious: one cell-centred pressure plus the six face velocities that its continuity equation uses. Our discretisation is *colocated* — all four fields live at every node — so the patch must be chosen deliberately.

Patch	Size	Comment
Node star	$1 + 27 \times 3 = 82$	one pressure, velocities of the 3^3 neighbourhood
Micro-element	$8 \times 4 = \mathbf{32}$	the 8 corners of one micro-cell, all fields
Pressure Schur only	8	micro-element patch with velocities eliminated

Recommendation: the micro-element patch

The micro-element patch is the right choice here for a reason specific to our code: the 32×32 patch operator $\mathcal{A}_k$ is *exactly the dense micro-cell matrix we already know how to build*. Passing identity slots to `cvfem_hex8_ns_upwind_jacobian_add_slots` writes it into a local buffer — the same trick `assemble_block_diag` uses and the same one the element-wise Galerkin assembly relies on. No new kernel is needed to form the patch operator.

It also aligns with every structure the solver already has: the micro-cell loop, the constant corner offsets on the lattice, and the deterministic two-pass scatter.

A subtlety in our favour: because Rhie–Chow supplies $C \neq 0$, the patch matrix has a non-zero pressure block and is nonsingular without any extra pinning. On an unstabilised saddle point the local systems are singular up to a constant and need care; we avoid that.

3 Variants, and what each costs

3.1 Full versus diagonal Vanka

Full Vanka inverts $\mathcal{A}_k$ exactly. *Diagonal* Vanka approximates the velocity block F_k by its diagonal, which lets the velocities be eliminated analytically and leaves a small pressure system — the local Schur complement

$$S_k = -C_k - B_k \operatorname{diag}(F_k)^{-1} B_k^\top, \quad (4)$$

of size 8×8 for the micro-element patch. This is the variant most production codes use; it retains the essential velocity–pressure coupling while removing most of the cost.

3.2 Cost model

At $L = 8$ with 108 macro-elements: 55 296 micro-cells, 60 625 nodes, 242 500 dof.

Variant	patch n	factor (flop)	apply (flop)	factors, all cells
Full Vanka	32	21 845	2048	432 MiB
Diagonal Vanka	8	341	128	27 MiB

Per-sweep cost of the local solves alone, against the current smoother:

nodal 4×4 block-Jacobi (current)	1.94 MFLOP
diagonal Vanka, 8×8 Schur	7.08 MFLOP
full Vanka, 32×32	113.25 MFLOP

The trade

The local solve is a small part of a sweep — the sweep is dominated by the residual evaluation, which is unchanged. Diagonal Vanka should raise sweep cost by tens of percent, not multiples. Against that, moving ρ from 0.98 to even 0.3 is a factor of ~ 50 in iteration count. Full Vanka’s 432 MiB of cached factors is the reason to start with the diagonal variant, not its flop count.

4 High-performance implementation on the semi-structured mesh

4.1 Additive versus multiplicative, and colouring

Equation (3) is *additive*: every patch reads the same residual, so all patches are independent and the sweep is embarrassingly parallel. It converges more slowly than the *multiplicative* form, in which each patch sees the updates of those before it (Gauss–Seidel in patches).

The multiplicative form parallelises by colouring patches so that same-coloured patches share no degrees of freedom. On our lattice this is exact and free: two micro-cells share a node iff their indices differ by at most one in every axis, so the parity classes

$$c(i, j, k) = (i \bmod 2) + 2(j \bmod 2) + 4(k \bmod 2) \in \{0, \dots, 7\} \quad (5)$$

give **eight colours**, and within a colour every pair of cells differs by at least two in some axis and is therefore independent. Eight sequential passes, each fully parallel over $L^3/8$ cells per macro-element.

4.2 Accumulation and determinism

Additive Vanka writes overlapping contributions to shared nodes — exactly the pattern the solver already handles. Reuse **SSScatter**: each macro-element writes its exclusively owned nodes directly and stages the shared ones, then a second pass sums each shared node’s contributions in a fixed order. No atomics, and the sweep is bitwise reproducible on any thread count, which the existing determinism gates will check for free.

4.3 Where the factorisations live

The patch matrices depend on the linearisation state, so they are valid for one Newton step and must be rebuilt when it changes — the same lifetime as the element-wise Galerkin coarse operators, and they can be built in the same pass.

- **Cache the 8×8 Schur factors** (27 MiB at $L = 8$): build once per Newton step, apply on every sweep. Six sweeps per V-cycle and hundreds of cycles per Newton step make this overwhelmingly the right trade.
- Store them in the *lattice* layout already used elsewhere — indexed by `(macro element, micro-cell)` — so the sweep streams them contiguously with no indirection, exactly as the 27-point stencil coarse operators do.
- Do *not* cache the full 32×32 factors; 432 MiB is not worth it before the diagonal variant has been measured.

4.4 Matrix-free, precisely

“Matrix-free” here means no *global* sparse matrix, not no local matrices. Vanka needs the patch entries, and we form them transiently per micro-cell from the existing kernel, use them, and discard them — which is what the element-wise Galerkin assembly already does. The fine-level residual evaluation, which dominates the sweep, stays exactly as it is.

5 Implementation plan

1. **Form the patch operator.** Per micro-cell, identity slots into `cvfem_hex8_ns_upwind_jacobian_add_slots` give the dense 32×32 $\mathcal{A}_k$. Already available; no new kernel.
2. **Build the local Schur complement.** Extract $\text{diag}(F_k)$, form $S_k = -C_k - B_k \text{diag}(F_k)^{-1} B_k^T$ (8×8), factor it, and store by `(element, cell)`. One pass per Newton step, fusable with the Galerkin assembly pass.
3. **Additive sweep first.** Residual, then per cell: gather 32 local values, solve S_k for the pressure correction, back-substitute the velocities, scatter through `SSScatter`. Additive is simpler and its correctness is easier to establish.
4. **Measure with SFEM.GMG.CHECK=3**, 150 sweeps, to the asymptote — the same gate that produced the 0.981 figure, so the comparison is like for like. Sweep ω .
5. **Then multiplicative with 8-colour parity** if the additive rate is insufficient. Expect a substantially better factor at the cost of eight sequential passes.
6. **Only then reconsider full Vanka**, and only if the diagonal variant’s rate is close enough to make the memory worth it.

Acceptance criterion

The measurement that matters is the standalone smoother rate at the *physical* Reynolds number, run to its asymptote with no coarse space. Block-Jacobi gives 0.981 at $\text{Re} = 1$. Any Vanka variant that does not comfortably beat that, at $\text{Re} \geq 100$, is not worth carrying — and a rate near or above 1 must be treated as disqualifying however good it looks over the first few sweeps, because the rate can fall for twenty sweeps and then climb through unity.

6 Risks and open questions

- **Colocated patches are not the classical construction.** Vanka’s rates are published for staggered discretisations. The micro-element patch is the natural colocated analogue, but the LFA numbers do not transfer automatically and our own measurement is the only evidence that will count.

- **Rhie–Chow coupling.** $C \neq 0$ makes the patches nonsingular, which helps, but the stabilisation scale $D_f = \text{rc } h^2 / (2\mu)$ grows as Re rises and as the mesh coarsens; the patch conditioning should be checked across the hierarchy, not only on the fine level.
- **Convection dominance.** All published rates cited here are for Stokes. Our failure is at $\text{Re} \geq 200$, where the convective term dominates, and no Stokes rate speaks to that directly. Measuring the Reynolds dependence of whatever smoother we adopt is not optional.
- **Coarse levels.** The coarse levels are assembled BSR, not matrix-free, so their patch operators must be extracted from the assembled matrix instead of from the cell kernel — a second, simpler code path.